Another Language - Python

Name: Rahul Vedant Pemsing
Student ID: 36305529
Date: 22/09/2025

Learning Journey and Evidence

Prior Knowledge and Experience

I am familiar with Python as after my A-levels, I had a lot of free time and had the chance to expand my python knowledge further. Therefore I am able to code terminal base programs in python without any issue.

Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

20/09/2025 - Read the python guide

Table of Contents

4. More Control Flow Tools
- 4.1. `if` Statements
- 4.2. `for` Statements
- 4.3. The `range()` Function
- 4.4. `break` and `continue` Statements
- 4.5. `else` Clauses on Loops
- 4.6. `pass` Statements
- 4.7. `match` Statements
- 4.8. Defining Functions
- 4.9. More on Defining Functions
  - 4.9.1. Default Argument Values
  - 4.9.2. Keyword Arguments
  - 4.9.3. Special parameters
    - 4.9.3.1. Positional-or-Keyword Arguments
    - 4.9.3.2. Positional-Only Parameters
    - 4.9.3.3. Keyword-Only Arguments
    - 4.9.3.4. Function Examples
    - 4.9.3.5. Recap
  - 4.9.4. Arbitrary Argument Lists
  - 4.9.5. Unpacking Argument Lists
  - 4.9.6. Lambda Expressions
  - 4.9.7. Documentation Strings
  - 4.9.8. Function Annotations
- 4.10. Intermezzo: Coding Style

Previous topic
3. An Informal Introduction to Python

Next topic
5. Data Structures

This page

4. More Control Flow Tools

As well as the `while` statement just introduced, Python uses a few more that we will encounter in this chapter.

4.1. `if` Statements

Perhaps the most well-known statement type is the `if` statement. For example:

```
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print('Negative changed to zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Single')
... else:
...     print('More')
...
More
```

There can be zero or more `elif` parts, and the `else` part is optional. The keyword '`elif`' is short for 'else if', and is useful to avoid excessive indentation. An `if` ... `elif` ... `elif` ... sequence is a substitute for the `switch` or `case` statements found in other languages.

If you're comparing the same value to several constants, or checking for specific types or attributes, you may also find the `match` statement useful. For more details see match Statements.

4.2. for Statements

Table of Contents

5. Data Structures
- 5.1. More on Lists
  - 5.1.1. Using Lists as Stacks
  - 5.1.2. Using Lists as Queues
  - 5.1.3. List Comprehensions
  - 5.1.4. Nested List Comprehensions
- 5.2. The `del` statement
- 5.3. Tuples and Sequences
- 5.4. Sets
- 5.5. Dictionaries
- 5.6. Looping Techniques
- 5.7. More on Conditions
- 5.8. Comparing Sequences and Other Types

Previous topic
4. More Control Flow Tools

Next topic
6. Modules

This page
Report a bug
Show source

5. Data Structures

This chapter describes some things you've learned about already in more detail, and adds some new things as well.

5.1. More on Lists

The list data type has some more methods. Here are all of the methods of list objects:

`list.append(x)`
Add an item to the end of the list. Similar to `a[len(a):] = [x]`.

`list.extend(iterable)`
Extend the list by appending all the items from the iterable. Similar to `a[len(a):] = iterable`.

`list.insert(i, x)`
Insert an item at a given position. The first argument is the index of the element before which to insert, so `a.insert(0, x)` inserts at the front of the list, and `a.insert(len(a), x)` is equivalent to `a.append(x)`.

`list.remove(x)`
Remove the first item from the list whose value is equal to *x*. It raises a `ValueError` if there is no such item.

`list.pop([i])`
Remove the item at the given position in the list, and return it. If no index is specified, `a.pop()` removes and

Table of Contents

6. Modules
- 6.1. More on Modules
  - 6.1.1. Executing modules as scripts
  - 6.1.2. The Module Search Path
  - 6.1.3. "Compiled" Python files
- 6.2. Standard Modules
- 6.3. The `dir()` Function
- 6.4. Packages
  - 6.4.1. Importing * From a Package
  - 6.4.2. Intra-package References
  - 6.4.3. Packages in Multiple Directories

Previous topic
5. Data Structures

Next topic
7. Input and Output

This page
Report a bug
Show source

6. Modules

If you quit from the Python interpreter and enter it again, the definitions you have made (functions and variables) are lost. Therefore, if you want to write a somewhat longer program, you are better off using a text editor to prepare the input for the interpreter and running it with that file as input instead. This is known as creating a *script*. As your program gets longer, you may want to split it into several files for easier maintenance. You may also want to use a handy function that you've written in several programs without copying its definition into each program.

To support this, Python has a way to put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a *module*; definitions from a module can be *imported* into other modules or into the *main* module (the collection of variables that you have access to in a script executed at the top level and in calculator mode).

A module is a file containing Python definitions and statements. The file name is the module name with the suffix `.py` appended. Within a module, the module's name (as a string) is available as the value of the global variable `__name__`. For instance, use your favorite text editor to create a file called `fibo.py` in the current directory with the following contents:

```
# Fibonacci numbers module

def fib(n):
    """Write Fibonacci series up to n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):
    """Return Fibonacci series up to n."""
```

From studying the Python guide, I learned several important aspects that differentiate Python from C++. Firstly, Python relies heavily on indentation rather than curly braces {} to define code blocks. This enforces readability but was initially something I had to be cautious about, as forgetting spaces could cause syntax errors. Secondly, Python uses dynamic typing, meaning that variables do not require explicit type declarations like int or double. Instead, the interpreter infers the type automatically. This saves time but also means I must be more aware of how variables are used to avoid type-related bugs.

Another key observation was that Python provides a rich standard library and many built-in functions. For example, operations such as finding the maximum, minimum, or average of a list can be done in one line. By contrast, in C++, I would usually need to write explicit loops or use additional libraries. Finally, I also reviewed how Python handles input/output and lists, which gave me confidence to proceed with the coding exercises in this project.

20/09/2025 - coding[Temperature tracker]

Description:

Asks the user to input a number of daily temperatures (array/list of doubles, up to 20) then output the following details:

i. Calculate and display the average temperature.
ii. Display the highest and lowest temperatures.
iii. Count how many days were above the average

My code in C/C++:

temp_tracker > temp.cpp > ...

```cpp
#include "splashkit.h"
#include <vector>

const int days_max = 20;

/**
 * @brief Reads a line of text input from the user.
 *
 * Prompts the user with the given message and then returns
 * the entered string.
 *
 * @param prompt The message shown to the user before input.
 * @return The string entered by the user.
 */
string read_string(string prompt){
    string line;
    write(prompt);
    line = read_line();
    return line;
}
```

```cpp
/**
 * @brief Reads and validates a double number from the user.
 *
 * Continues to prompt until the user enters a valid number.
 *
 * @param prompt The message shown to the user before input.
 * @return The number entered by the user as a double.
 */
double read_double(string prompt)
{
    string line;
    line = read_string(prompt);
    while (!is_double(line))
    {
        write_line("Please enter a valid number");
        line = read_string(prompt);
    }
    return convert_to_double(line);
}

/**
 * @brief Prompts the user to enter a temperature value.
 *
 * @return The temperature entered by the user.
 */
double read_data()
{
    return read_double("Enter the temperature: ");
}
```

```cpp
/**
 * @brief Displays the maximum and minimum values in an array.
 *
 * Iterates through the readings and finds the largest and smallest values.
 *
 * @param array The vector of readings.
 * @param days_readings The number of readings to process.
 */
void display_max_min(const std::vector<double> &array, const int &days_readings)
{
    double max = array[0];
    double min = array[0];
    for (int i = 0; i < days_readings; i++)
    {
        if (array[i] > max)
        {
            max = array[i];
        }
        if (array[i] < min)
        {
            min = array[i];
        }
    }
    write_line("Max: " + std::to_string(max));
    write_line("Min: " + std::to_string(min));
}
```

My code in Python:

```python
days_max = 20

def read_double(prompt):
    """Read and return a valid float number"""
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Please enter a valid number")

def read_data():
    """Read data from user"""
    return read_double("Enter the temperature: ")

def display_max_min(array):
    """Display max and min temperature"""
    if len(array) == 0:
        print("No data entered")
        return
    print(f"Max: {max(array)}")
    print(f"Min: {min(array)}")

def average(array, day_readings):
    """Return the avarage temperature of the entered temperature"""
    return sum(array) / day_readings

def above_average(array, day_readings):
    """Display any temperature that is above the average temperature"""
    avg = average(array, day_readings)
    for temp in array:
        if temp > avg:
            print(temp)
```

```python
def main():
    days_readings = 0
    choose = "y"
    array = []
    while choose != "n" and days_readings<20:
        array.append(read_data())
        choose = input("Do you want to enter the number of readings? (y/n): ").lower()
        days_readings += 1
        if days_readings == 20:
            print("You have reached the maximum number of readings.")
            break
    print()
    print(f"The average is {average(array, days_readings)}")
    print()
    display_max_min(array)
    print()
    print("Above average: ")
    above_average(array, days_readings)


if __name__ == "__main__":
    main()
```

Output python:

```
$ C:/msys64/mingw64/bin/python.exe c:/Users/Vedant/Desktop/mycode/my_python_code/temp_tracker/temp.py
Enter the temperature: 15
Do you want to enter the number of readings? (y/n): y
Enter the temperature: 60
Do you want to enter the number of readings? (y/n): y
Enter the temperature: 80
Do you want to enter the number of readings? (y/n): n

The average is 51.666666666666664

Max: 80.0
Min: 15.0

Above average:
60.0
80.0
```

```cpp
/**
 * @brief Calculates the average of an array of values.
 *
 * @param array The vector of readings.
 * @param days_readings The number of readings to process.
 * @return The average value.
 */
double average(const std::vector<double> &array, const int &days_readings)
{
    double avg = 0;
    for (int i = 0; i < days_readings; i++)
    {
        avg += array[i];
    }
    avg /= days_readings;
    return avg;
}

/**
 * @brief Displays all values above the average.
 *
 * Iterates through the readings and prints any value greater than the average.
 *
 * @param array The vector of readings.
 * @param days_readings The number of readings to process.
 */
void above_average(const std::vector<double> &array, const int &days_readings)
{
    double avg = average(array, days_readings);
    for (int i = 0; i < days_readings; i++)
    {
        if (array[i] > avg)
        {
            write_line(std::to_string(array[i]));
        }
    }
```

```cpp
/**
 * @brief Program entry point.
 *
 * Allows the user to enter up to 20 temperature readings.
 * After input is complete, it calculates and displays:
 * - The average value
 * - The maximum and minimum values
 * - All values above the average
 *
 * @return 0 when the program exits successfully.
 */
int main()
{
    int days_readings = 0;
    string choose = "y";
    std::vector<double> array;

    while (choose != "n" && days_readings < 20)
    {
        array.push_back(read_data());
        choose = read_string("Do you want to enter the number of readings? (y/n): ");
        days_readings++;
        if (days_readings == 20)
        {
            write_line("You have reached the maximum number of readings.");
            break;
        }
    }
```

```cpp
    write_line("");
    write("The average is ");
    write_line(std::to_string(average(array, days_readings)));
    write_line("");
    display_max_min(array, days_readings);
    write_line("");
    write_line("Above average: ");
    above_average(array, days_readings);
    return 0;
}
```

Output C/C++:

```
Enter the temperature: 15
Do you want to enter the number of readings? (y/n): y
Enter the temperature: 60
Do you want to enter the number of readings? (y/n): y
Enter the temperature: 80
Do you want to enter the number of readings? (y/n): n

The average is 51.666667

Max: 80.000000
Min: 15.000000

Above average:
60.000000
80.000000
```

Comparison between C/C++ and python:

In C++, I had to manually calculate the sum, maximum, and minimum using loops. By contrast, Python allowed me to do this in a few lines by using built-in functions like sum(), max(), and min(). The Python version is not only shorter but also easier to understand at first glance. Another difference is memory management. In C++, I had to declare a fixed-size array (double temps[20]), whereas Python's list

structure grows dynamically. This means Python code is more flexible when dealing with different numbers of inputs.

What I learned from this:
This task showed me how Python reduces boilerplate code and helps me focus on the logic rather than technical details like memory allocation. It also reinforced the idea that Python is excellent for quick prototyping and data processing.

20/09/2025 - coding[Grocery List Manager]

Description:
Allow the user to build a grocery list (array/list of strings, up to 20). The user should be able to:

i. Add items to the list
ii. Remove items from the list
iii. View the list

My code in C/C++:

grocery_list_manager > g_list.cpp > ...

```cpp
#include "splashkit.h"
#include <vector>

const int max_item = 20;

/**
 * @brief Prompts the user for a string input.
 *
 * @param prompt The message to display before input.
 * @return The string entered by the user.
 */
string read_string(string prompt){
    string line;
    write(prompt);
    line = read_line();
    return line;
}
```

```cpp
/**
 * @brief Reads and validates an integer within a given range.
 *
 * Keeps asking until the user enters a valid integer between `min` and `max`.
 *
 * @param prompt The message to display before input.
 * @param min The minimum valid value.
 * @param max The maximum valid value.
 * @return The validated integer entered by the user.
 */
int read_int_limit(string prompt, int min, int max){
    string line = read_string(prompt);
    while (!is_integer(line) || stoi(line) < min || stoi(line) > max){
        write_line("Please enter a valid number");
        line = read_string(prompt);
    }
    return stoi(line);
}
```

My code in Python:

Grocery_List_Manager > d_list.py > main

```python
max_item = 20

def read_integer_limit(prompt,min,max):
    """Prompt the user for an integer input within a given range."""
    flag = True
    while flag:
        try:
            user_input = int(input(prompt))
            if user_input>= min and user_input <= max:
                flag = False
                return user_input
        except ValueError:
            flag= True
            print("Error: Invalid input. Please enter an integer.")

def add_item(array):
    """Add a new item to the grocery list."""
    item = input("Enter item: ")
    array.append(item)
    print("Item added")

def display_items(array):
    """Display all items currently in the grocery list."""
    print("Items in the grocery list: ")
    for item in array:
        print(item)
```

```python
def remove_item(array):
    """Remove an item from the grocery list if it exists."""
    item = input("Enter item to remove: ")
    if item in array:
        array.remove(item)
        print("Item removed")
    else:
        print("Item not found")

def menu():
    """Display the main menu options for the grocery list manager."""
    print("1. Add item")
    print("2. Display items")
    print("3. Remove item")
    print("4. Quit")
```

```cpp
/**
 * @brief Adds a new item to the grocery list.
 *
 * @param items The vector storing grocery items.
 */
void add_item(std::vector<string> &items){
    string item;
    item = read_string("Enter item: ");
    items.push_back(item);
    write_line("Item added");
}

/**
 * @brief Displays all items in the grocery list.
 *
 * If the list is empty, it will display a message instead.
 *
 * @param items The vector storing grocery items.
 */
void print_items(std::vector<string> &items){
    if (items.size() == 0){
        write_line("List is empty");
        return;
    }
    write_line("Items in the grocery list: ");
    for(int i = 0; i < items.size(); i++){
        write_line("-> " + items[i]);
    }
}
```

```cpp
/**
 * @brief Removes an item from the grocery list by name.
 *
 * If the item exists, it is removed. Otherwise, a "not found" message is shown.
 *
 * @param items The vector storing grocery items.
 */
void remove_item(std::vector<string> &items){
    string item;
    item = read_string("Enter item to remove: ");
    for(int i = 0; i < items.size(); i++){
        if(items[i] == item){
            items.erase(items.begin() + i);
            write_line("Item removed");
            return;
        }
    }
    write_line("Item not found");
}

/**
 * @brief Displays the main menu options.
 */
void menu(){
    write_line("1. Add item");
    write_line("2. Print items");
    write_line("3. Remove item");
    write_line("4. Quit");
}
```

```cpp
/**
 * @brief Entry point for the Grocery List Manager program.
 *
 * Allows the user to:
 * - Add up to 20 items
 * - Print all items
 * - Remove items
 * - Quit the program
 *
 * @return 0 on successful program termination.
 */
int main(){
    std::vector<string> items;
    int curr_items = 0;
    int choice;
    do{
        menu();
        choice = read_int_limit("Enter choice: ",1,4);
        write_line("");
        switch (choice)
        {
        case 1:
            if (curr_items<max_item){
                add_item(items);
                write_line("");
                curr_items++;
            }else
            {
                write_line("List is full");
            }
            break;
```

```python
def main():
    array = []
    item = 0
    menu()
    choice = read_integer_limit("Enter your choice: ",1,4)
    while choice != 4:
        if choice == 1:
            print("")
            if item < max_item:
                item += 1
                add_item(array)
            else:
                print("List is full")
            print("")
        elif choice == 2:
            print("")
            display_items(array)
            print("")
        elif choice == 3:
            print("")
            remove_item(array)
            print("")

        menu()
        choice = read_integer_limit("Enter your choice: ", 1, 4)

if __name__ == "__main__":
    main()
```

Output python:

```
        case 2:
            write_line("");
            print_items(items);
            write_line("");
            break;
        case 3:
            write_line("");
            remove_item(items);
            write_line("");
            break;
        }

    }while(choice != 4);
    return 0;
}
```

Output C/C++:

```
$ C:/msys64/mingw64/bin/python.exe
1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 1

Enter item: milk
Item added

1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 1

Enter item: water
Item added

1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 2

Items in the grocery list:
milk
water

1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 3

Enter item to remove: milk
Item removed
```

```
1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 1

Enter item: milk
Item added

1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 1

Enter item: water
Item added

1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 2


Items in the grocery list:
-> milk
-> water

1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 3


Enter item to remove: milk
Item removed
```

```
1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 3

Enter item to remove: butter
Item not found

1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 2

Items in the grocery list:
water

1. Add item
2. Display items
3. Remove item
4. Quit
Enter your choice: 4
```

```
1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 3


Enter item to remove: butter
Item not found

1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 2


Items in the grocery list:
-> water

1. Add item
2. Print items
3. Remove item
4. Quit
Enter choice: 4
```

Comparison between C/C++ and python:
In C++, I had to use a vector to hold the items and write a loop to remove elements by comparing them. In Python, list operations such as append() and remove() make this much simpler. The Python version required fewer lines of code, and the control flow was clearer. Additionally, Python's for loop is more intuitive when iterating over items, while C++ requires iterator manipulation.

What I learned from this:
This exercise taught me how much simpler user input and list operations are in Python. The Python version feels natural and concise, while the C++ version feels more rigid.

21/09/2025 - coding[Student Grade Analyser]

Description:

Allow the user to enter a list of grade results (array/list of integers, up to 20). Output the average grade, pass rate (% that are >= 50/100) and number of high distinction achievements (number >= 80/100)

My code in C/C++:

```cpp
#include "splashkit.h"
#include <vector>

const int max_student = 20;

/**
 * @brief Reads a string from user input.
 *
 * @param prompt The message shown to the user.
 * @return The entered string.
 */
string read_string(string prompt){
    string line;
    write(prompt);
    line = read_line();
    return line;
}
```

```cpp
/**
 * @brief Reads and validates an integer within a given range.
 *
 * Keeps asking until the user enters an integer between min and max.
 *
 * @param prompt The message to display before input.
 * @param min The minimum valid value.
 * @param max The maximum valid value.
 * @return A validated integer.
 */
int read_int_limit(string prompt, int min, int max){
    string line = read_string(prompt);
    while (!is_integer(line) || stoi(line) < min || stoi(line) > max){
        write_line("Please enter a valid number");
        line = read_string(prompt);
    }
    return stoi(line);
}
```

```cpp
/**
 * @brief Adds a new student grade to the grade list.
 *
 * Prompts for a grade (0-100) and stores it in the vector.
 *
 * @param grades A vector storing all student grades.
 */
void add_stu_grade(vector<int> &grades){
    int grade;
    grade = read_int_limit("Enter the grade: ", 0, 100);
    grades.push_back(grade);
    write_line("Grade added");
}

/**
 * @brief Calculates and prints the average grade.
 *
 * @param grades A vector storing all student grades.
 * @param no_stu Number of students.
 */
void avg_grade(vector<int> &grades, int no_stu){
    int total = 0;
    for (int i = 0; i < no_stu; i++){
        total += grades[i];
    }
    write_line("The average grade is: " + std::to_string(total/no_stu));
}
```

My code in Python:

```python
max_student = 20

def read_integer_limit(prompt, min_val, max_val):
    """Prompt the user for an integer input within a given range."""
    while True:
        try:
            user_input = int(input(prompt))
            if min_val <= user_input <= max_val:
                return user_input
            print("Please enter a valid number")
        except ValueError:
            print("Please enter a valid number")

def add_stu_grade(grades):
    """Add a student grade to the list."""
    grade = read_integer_limit("Enter the grade: ", 0, 100)
    grades.append(grade)
    print("Grade added")

def avg_grade(grades, no_stu):
    """Print the average grade of the students."""
    total = sum(grades[:no_stu])
    print(f"The average grade is: {total // no_stu}")

def pass_rate(grades, no_stu):
    """Print the percentage of students who passed (grade >= 50)."""
    pass_count = sum(1 for grade in grades[:no_stu] if grade >= 50)
    rate = (pass_count / no_stu) * 100
    print(f"The pass rate is: {rate}")

def high_disc_rate(grades, no_stu):
    """Print the percentage of students with high distinction (grade >= 80)."""
    disc_count = sum(1 for grade in grades[:no_stu] if grade >= 80)
    rate = (disc_count / no_stu) * 100
    print(f"The high distinction rate is: {rate}")
```

```python
def main():
    grades = []
    no_stu = read_integer_limit("Enter the number of students: ", 1, max_student)

    for _ in range(no_stu):
        add_stu_grade(grades)


    avg_grade(grades, no_stu)
    print()
    pass_rate(grades, no_stu)
    print()
    high_disc_rate(grades, no_stu)
    print()

if __name__ == "__main__":
    main()
```

Output python:

```
Enter the number of students: 3
Enter the grade: 55
Grade added
Enter the grade: 13
Grade added
Enter the grade: 90
Grade added
The average grade is: 52

The pass rate is: 66.66666666666666

The high distinction rate is: 33.33333333333333
```

```cpp
/**
 * @brief Calculates and prints the pass rate (grades >= 50).
 *
 * @param grades A vector storing all student grades.
 * @param no_stu Number of students.
 */
void pass_rate(vector<int> &grades, int no_stu){
    int pass = 0;
    for (int i = 0; i < no_stu; i++){
        if (grades[i] >= 50){
            pass +=1;
        }
    }

    write_line("p= " + std::to_string(pass));
    double pass_rate = convert_to_double(std::to_string(pass))/convert_to_double(std::to_string(no_stu)) * 100;
    write_line("The pass rate is: " + std::to_string(pass_rate));
}

/**
 * @brief Calculates and prints the high distinction rate (grades >= 80).
 *
 * @param grades A vector storing all student grades.
 * @param no_stu Number of students.
 */
void high_disc_rate(vector<int> &grades, const int &no_stu){
    int disc = 0;
    for (int i = 0; i < no_stu; i++){
        if (grades[i] >= 80){
            disc +=1;
        }
    }
    write_line("hd= " + std::to_string(disc));
    double high_disc = convert_to_double(std::to_string(disc))/convert_to_double(std::to_string(no_stu)) * 100;
    write_line("The high distinction rate is: " + std::to_string(high_disc));
}
```

```cpp
/**
 * @brief Entry point for the student grade analyzer program.
 *
 * Prompts for the number of students, collects grades, and prints:
 * - The average grade
 * - The pass rate
 * - The high distinction rate
 */
int main(){
    vector<int> grades;
    int no_stu;
    no_stu = read_int_limit("Enter the number of students: ", 1, max_student);
    for (int i = 0; i < no_stu; i++){
        add_stu_grade(grades);
    }
    write_line("stu: " + std::to_string(no_stu));
    write_line("");
    avg_grade(grades, no_stu);
    write_line("");
    pass_rate(grades, no_stu);
    write_line("");
    high_disc_rate(grades, no_stu);
    write_line("");
    return 0;
}
```

Output C/C++:

```
Enter the number of students: 3
Enter the grade: 55
Grade added
Enter the grade: 13
Grade added
Enter the grade: 90
Grade added
stu: 3

The average grade is: 52

p= 2
The pass rate is: 66.666667

hd= 1
The high distinction rate is: 33.333333
```

Comparison between C/C++ and python:

In C++, I had to explicitly maintain counters for passing students and high distinctions. In Python, I could simply use list comprehensions like sum(1 for g in grades if g >= 50). This makes the Python version shorter and easier to extend. Memory management in Python is automatic, while in C++ I had to allocate a fixed-size array (grades[20]).

What I learned from this:

Python provides elegant ways to handle data analysis tasks with concise syntax. It also highlighted how Python's strengths lie in readability and rapid development.

Reflection

What is the most important thing you learned from this and why?

The most important thing I learned is that while both C++ and Python can solve the same problems, Python emphasizes simplicity and readability. This is important because, in real-world programming, maintainability and clarity are just as critical as performance.

How quickly were you able to learn the basics of Python?

I was able to learn Python basics within a few days. My prior knowledge of python gave me a good continuation base and my knowledge in C++ gave me a strong foundation in programming logic, so I only needed to adapt to Python's syntax and conventions.

What were the trickiest parts of getting started with Python?

The trickiest part was getting used to Python's use of indentation to define code blocks. At first, I would accidentally misalign code, which caused errors. Another tricky aspect was adjusting to Python's dynamic typing, which requires me to think differently about variable handling compared to statically typed C++.

Where and how did your understanding of the programming concepts help you with this?

It helps me very much as understanding programming concepts is not limited by which programming language I am using, it applies to almost every language. So only the syntax of the new language needs to be learned so be able to use those concepts.

Of course some concepts will be less used than others, for example creating my own dynamic array in C++ was tricky but in python, List is inherently dynamic, so there is no need to build my own dynamic list.

That said, having knowledge on as many as possible concepts will make me a better programmer since overreliance on existing libraries is not a good thing.

References

Generative AI acknowledgement

No AI was used.